



UNIVERSITÀ DEGLI STUDI DI MILANO

Sicurezza & Privacy

GPG / Web Of Trust

Andrea Monzani

Prof. Danilo Bruschi



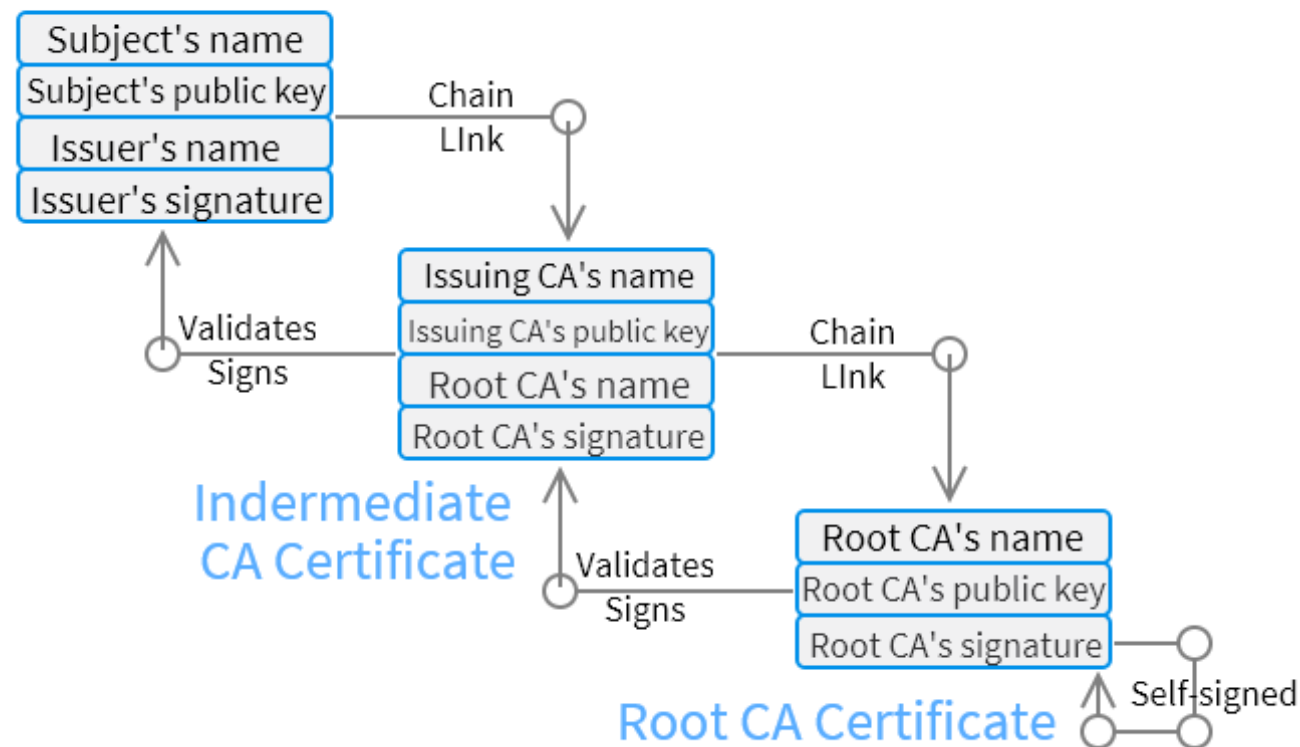


Certification Methods

- **Certification Authorities (CAs)**: as defined in the X.509 standard, operate within a **hierarchical trust model** in which they serve as the foundational root of trust
 - Part of what is called a **Public Key Infrastructure** (PKI) scheme
- **Pretty Good Privacy (PGP)**: does not depend on centralized root-of-trust authorities; instead, it is built on **trust relationships between individual users**

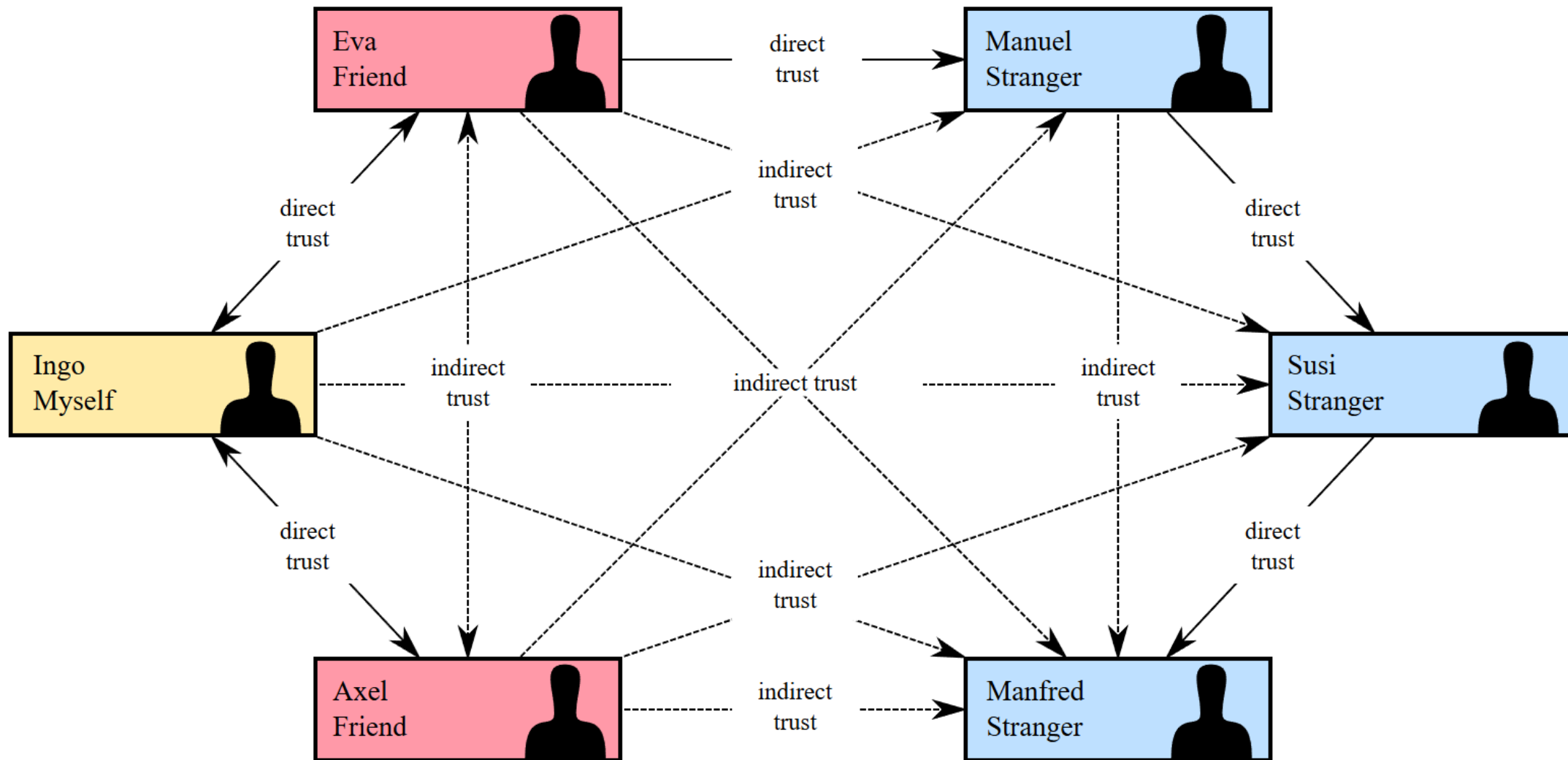
PKI vs GPG

End-entity Certificate



- In a PKI system, if a Trusted Root CA is compromised, an attacker could issue new certificates for any domain, and those certificates would be accepted as valid

PKI vs GPG



GPG

- GNU Privacy Guard (GnuPG or GPG) is a replacement for PGP software suite
- Compliant with the OpenPGP standard
- Installation:
<https://gnupg.org/download/#binary>

OS	Where	Description
Linux	download sig	<i>GnuPG Desktop</i> ® ApplImage with the current <i>GnuPG</i>
Windows	Gpg4win	Full featured Windows version of <i>GnuPG</i>
	download sig	Simple installer for the current <i>GnuPG</i>
	download sig	Simple installer for <i>GnuPG 1.4</i>
OS X	Mac GPG	Installer from the gpgtools project
	GnuPG for OS X	Installer for <i>GnuPG</i>
Debian	Debian site	GnuPG is part of Debian
RPM	rpmfind	RPM packages for different OS
Android	Guardian project	Provides a GnuPG framework
VMS	antinode.info	A port of GnuPG 1.4 to OpenVMS
RISC OS	home page	A port of GnuPG to RISC OS



The concept of trust

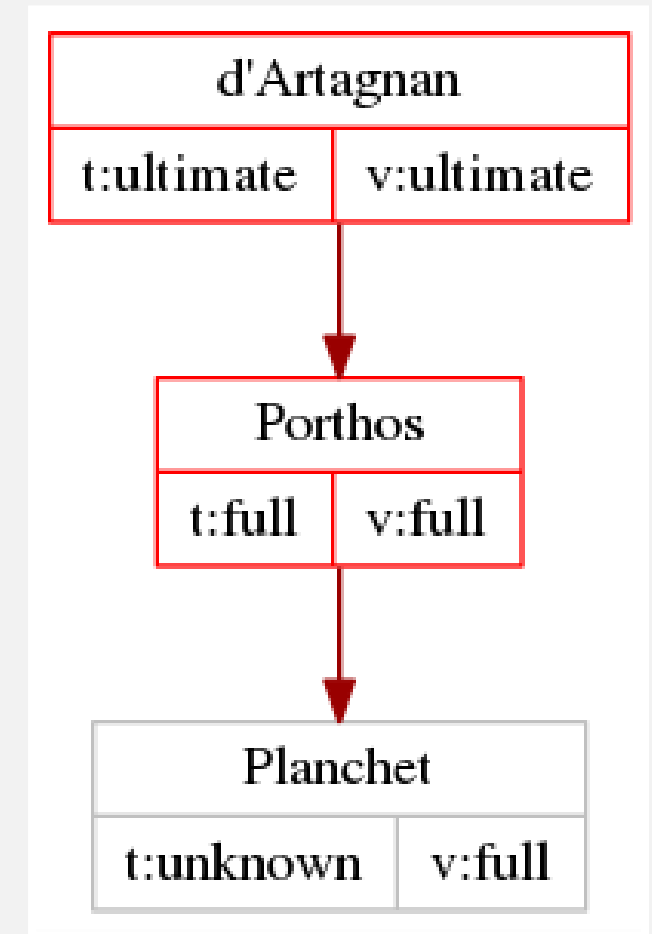
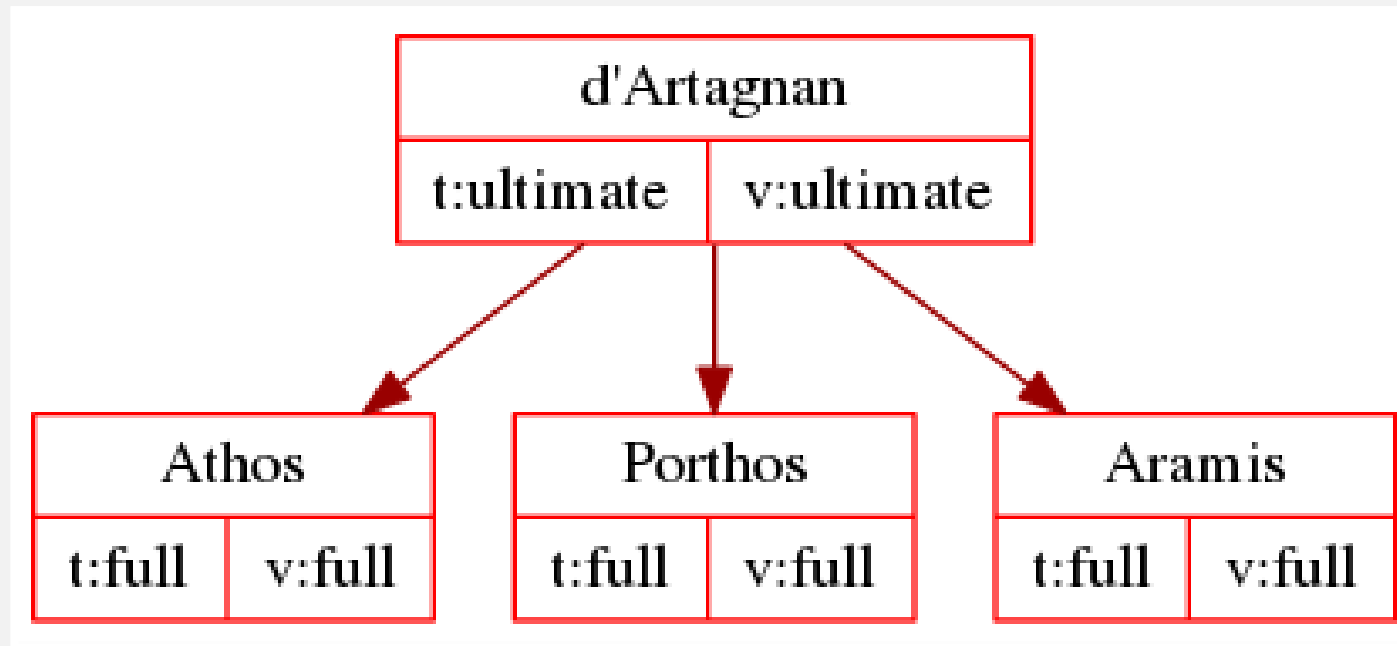
- In PKI, *trust* is the certainty that the public key we have for the entity X actually belongs to X
 - Ensured through a chain of signatures that ultimately terminates at a (usually) hardcoded, trusted certification authority
- In PGP, *trust* reflects how much **one entity trusts another to accurately verify and guarantee for other people's identities**



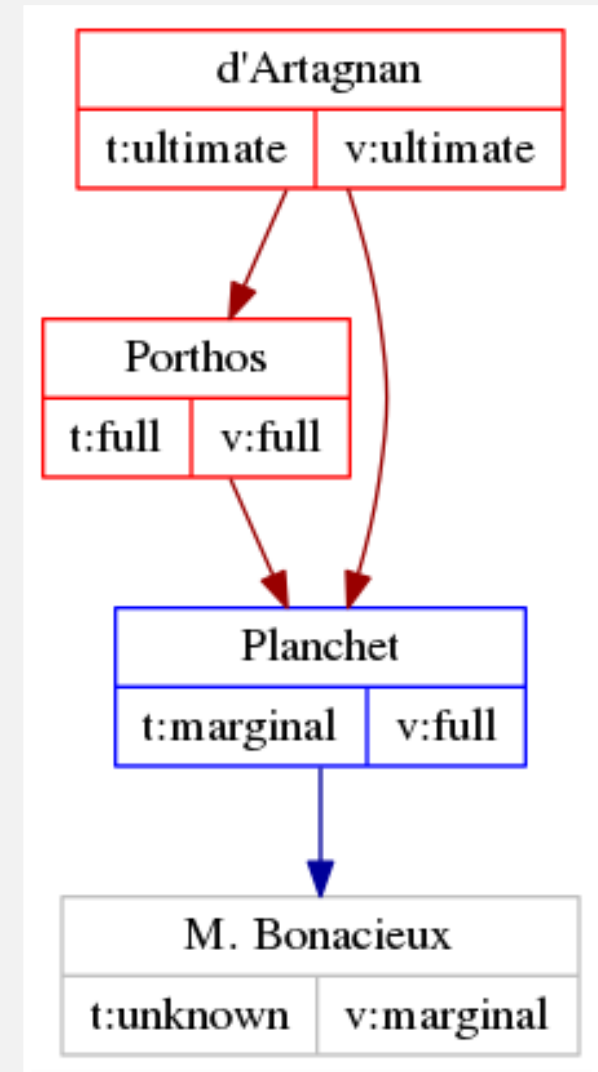
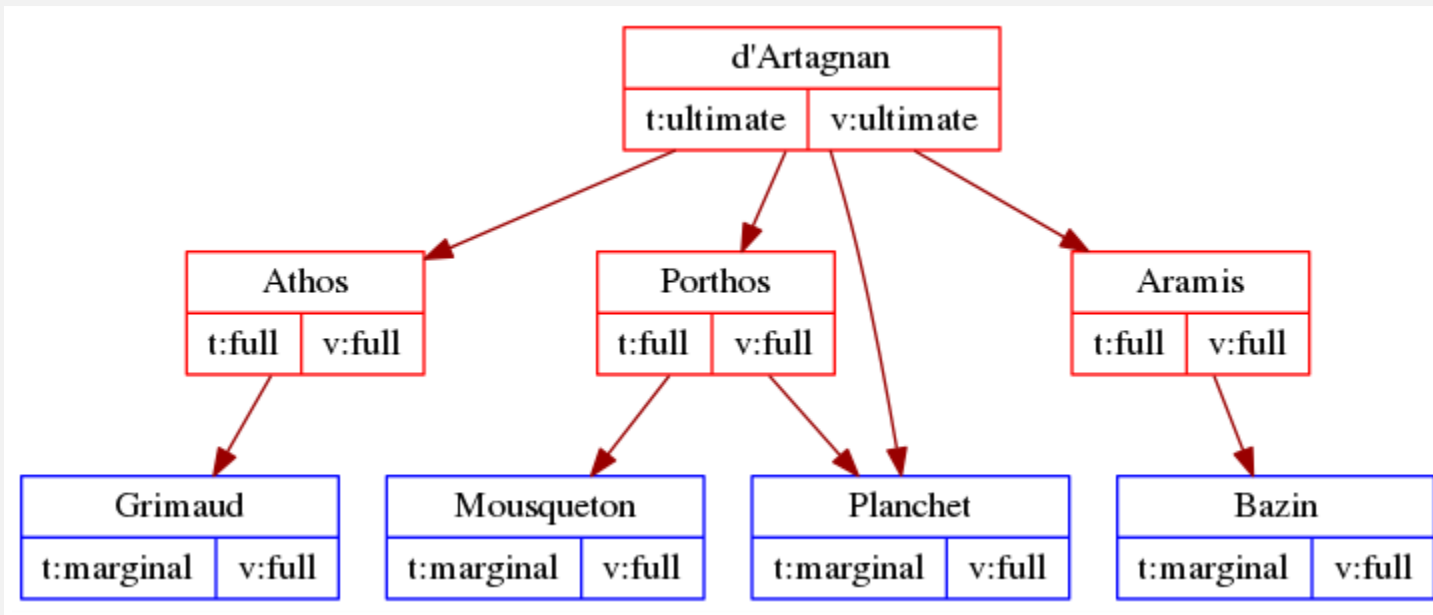
Trust and Validity

- Key **validity** is the certainty we have that the key in question belongs to the person with whom we want to communicate. **It is calculated based on owner trust and the number of signatures on the key**
- Key **trust** must be **set by you**, the user, for each key in your keyring before it is included in the web of trust calculations
- Trust can be full or marginal. It takes only one fully trusted signature to mark a key as “valid” but at least 3 marginally trusted keys to do the same.

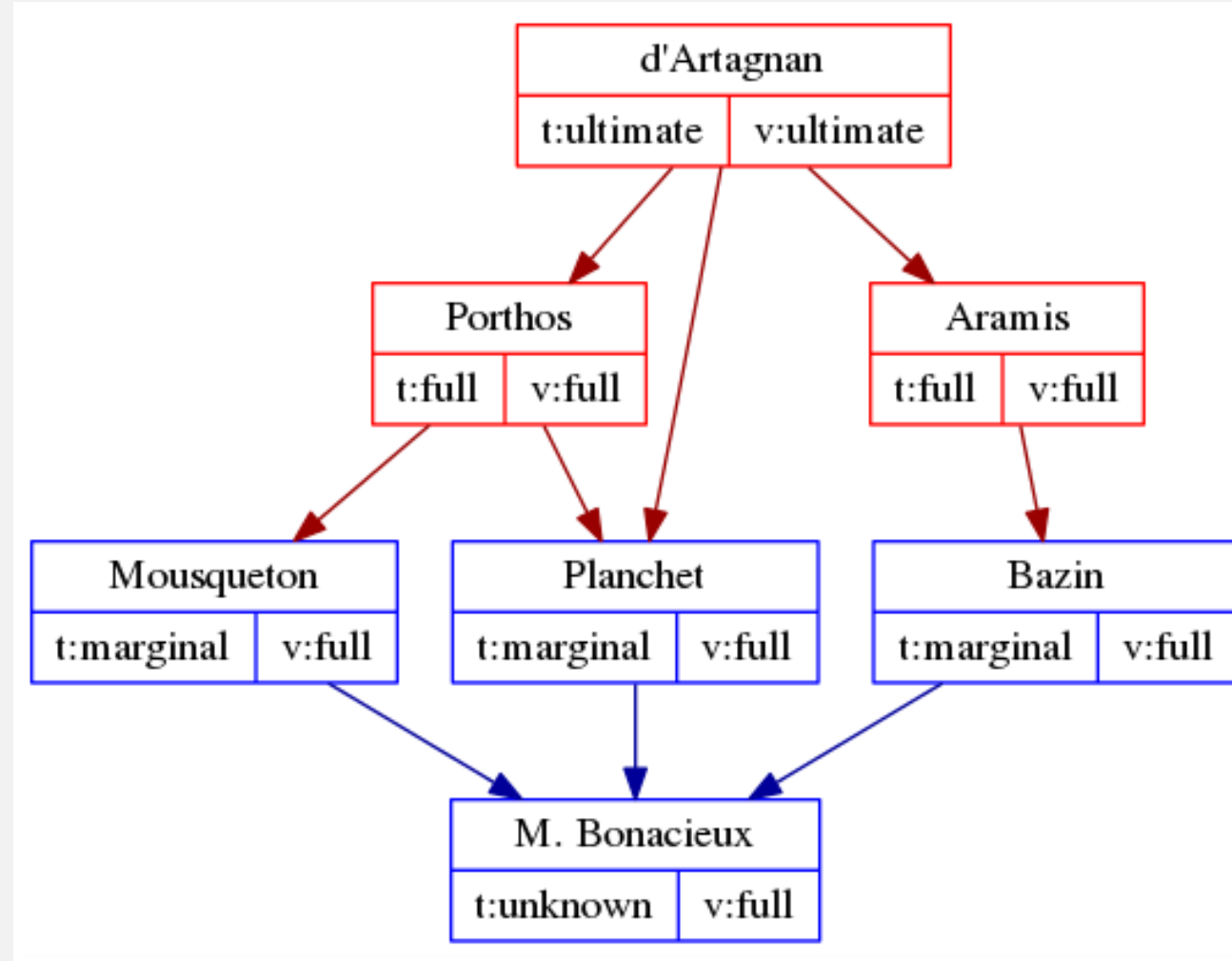
Trust and Validity



Trust and Validity



Trust and Validity





GPG – Encrypt Data (symmetric)

Encryption:

```
gpg [--output <name>] --symmetric [--cipher-algo <algo> [--armor] <input>
```

Decryption

```
gpg [--output <name>] -d <data>
```



GPG – Key Generation

```
gpg --generate-key
```

```
gpg --gen-key
```

Quick and easy way to generate a key, only name, email and password needed

```
gpg --full-generate-key
```

```
gpg --full-gen-key
```

Complete key generation, both algorithm, key length and expiration date can be specified

GPG – Key Listing

`gpg --list-keys / gpg -k`

Lists all the keys in the keyring

```
~/SP/Lab5$ gpg --list-keys
pub      rsa3072 2025-11-16 [SC] [expires: 2027-11-16]
          9A53001FC4B426288F1CC10BA0C0A95729C8BE2C
uid                               [ultimate] SicurezzaEPrivacy_1
sub      rsa3072 2025-11-16 [E] [expires: 2027-11-16]
```

Fingerprint

GPG – Secret Key Listing

```
gpg --list-secret-keys
```

Lists all the secret keys in the keyring

```
~/SP/Lab5$ gpg --list-secret-keys
sec    rsa3072 2025-11-16 [SC] [expires: 2027-11-16]
        9A53001FC4B426288F1CC10BA0C0A95729C8BE2C
uid                    [ultimate] SicurezzaEPrivatezza_1
ssb    rsa3072 2025-11-16 [E] [expires: 2027-11-16]
```

GPG – Key Inspection

`gpg --edit-key <key>`

Shows additional information w.r.t. `--list-keys` such as trust, and can be used to perform additional operations such as adding UIDS, subkeys, setting the trust, signing the key

```
~/SP/Lab5$ gpg --edit-key 9A53001FC4B426288F1CC10BA0C0A95729C8BE2C
Secret key is available.

sec  rsa3072/A0C0A95729C8BE2C
     created: 2025-11-16  expires: 2027-11-16  usage: SC
     trust: ultimate      validity: ultimate
ssb  rsa3072/019189DAFD5EEFA7
     created: 2025-11-16  expires: 2027-11-16  usage: E
[ultimate] (1). SicurezzaEPrivatezza_1
```

Key ID



GPG – Importing/Exporting Keys

```
gpg [--output <keyname>] --export [--armor] <UID/Fingerprint>
```

Export the specified key, if --output is specified the key will be written in a file, otherwise it will be written on the screen.

The --armor flag uses ascii format to represent it.

```
gpg --import <filename>
```

Import the key present in the specified file in the user keyring

GPG – Asymmetric Encryption

```
gpg [--output <name>] -e [-r <recipient>] [--armor] <data>
```

```
gpg [--output <name>] --encrypt [-r <recipient>] [--armor] <data>
```

Encrypt specified data with the recipient key

```
gpg [--output <name>] [--local-user <UID/Fingerprint>] -d <data>
```

Decrypt specified data using the default key, or the one specified using --local-user flag

GPG – Sign Data

```
gpg --sign [--local-user <UID/Fingerprint>] [--armor] <data>
```

```
gpg --clearsign [-u <UID/Fingerprint>] [--armor] <data>
```

```
gpg --detach-sign [-u <UID/Fingerprint>] [--armor] <data>
```

Signs the specified data using the given key, with the `--clearsign` flag no compression take place and the result is legible, whereas with `--detach-sign` the signature is written in a different file. The `--armor` flag uses ascii format to represent it.

```
gpg --verify
```



GPG – Sign Keys

```
gpg --sign-key [-u <UID/Fingerprint>] <KeyId/Fingerprint>
```

Signs the specified key using the given key.

```
gpg --list-signs <key>
```

Lists all the keys that signed a specific key in the keyring

GPG – Interact with a keyserver

```
gpg --keyserver <url> --search-keys <UID/Fingerprint/KeyId>
```

Search for the keys matching the UID/Fingerprint in the specified keyserver

```
gpg --keyserver <url> --send-keys <UID/Fingerprint/KeyId>
```

Sends the key to the specified keyserver so that others can easily look for it

```
gpg --keyserver <url> --recv-keys <UID/Fingerprint/KeyId>
```

Searches for the specified key in the keyserver and adds it to the keyring



References

- [PGP Web of Trust: Core Concepts Behind Trusted Communication](#)